

SAOWEII CARS - Retro Racing Game

Overview

SAOWEII CARS is a classic arcade-style racing game written entirely in x86 Assembly language for DOS. The game runs in VGA Mode 13h (320x240, 256 colors) and features a complete racing experience with obstacles, collectibles, music, and a persistent high score system.

Core Features & Assembly Concepts

1. Graphics System

- **VGA Mode 13h Programming:** Direct video memory manipulation using segment 0xA000
- **Pixel-Perfect Rendering:** Custom sprite drawing routines with transparency support (color 255 = transparent)
- **Double Buffering Simulation:** VSync synchronization (DrawBackground, WaitVSync) to prevent screen tearing
- **Custom Font Rendering:** Bitmap font system with 5x7 pixel character patterns for A-Z, 0-9, and special characters
- **Sprite Library:** Multiple sprites including:
 - Player car (12x14 pixels)
 - Enemy vehicles (12x14 pixels with fade-in effect)
 - Large title car (24x28 pixels)
 - Environmental elements (trees, flowers, rocks)
 - Animated spark effects (3 frames, 7x7 pixels)
 - UI elements (coins, fuel canisters)

2. Game States & Flow Control

- **State Machine Architecture:** Four game states (Start=0, Playing=1, GameOver=2, End=3)
- **Conditional Jumps:** Extensive use of je, jne, jb, ja for state transitions
- **Structured Programming:** Organized main loop with clear state checking

3. Player Movement & Input

- **BIOS Keyboard Interrupt (INT 16h):**
 - Function 01h: Check for keystroke without waiting
 - Function 00h: Read keystroke from buffer
- **Arrow Key Detection:** Scan code comparison (4Bh=Left, 4Dh=Right, 48h=Up, 50h=Down)
- **Boundary Checking:** Collision detection with screen edges (X: 85-216, Y: 10-172)
- **Smooth Movement:** 8-pixel increments with sprite erasure and redraw

4. Advanced Music System

- **Timer Interrupt Hooking (INT 08h):** Custom ISR replaces default system timer
- **PC Speaker Programming:** Direct port I/O (ports 42h, 43h, 61h) for tone generation
- **Frequency Calculation:** Formula: $\text{divisor} = 1193180 / \text{frequency}$
- **Multi-Track Audio:**
 - Background music (looping melody)
 - Sound effects (coin, fuel, crash) with priority system
 - Music track switching (gameplay → game over)
- **Note Duration System:** Tick-based timing with configurable note lengths
- **ISR Chain Preservation:** Saves and restores original timer vector on exit

5. Collision Detection

- **Bounding Box Algorithm:** Rectangle intersection tests for:
 - Player vs. Enemies (20x24 pixel tolerance)
 - Player vs. Coins (22x22 pixel tolerance)
 - Player vs. Fuel (22x24 pixel tolerance)
- **Pixel-Perfect Hit Detection:** Uses sprite dimensions for accurate collision
- **Spawn Overlap Prevention:** Checks all active entities before spawning new ones

6. Dynamic Entity Management

- **Array-Based Object Pools:**
 - Enemies: 5 concurrent slots with fade-in states
 - Coins: 3 concurrent slots
 - Fuel items: 2 concurrent slots
- **Parallel Arrays:** Separate X, Y, Active, and FadeState arrays indexed by entity ID

- **Circular Spawn System:** Timer-based spawning (40 ticks for enemies, 80 for coins, 120 for fuel)
- **Lane-Based Positioning:** Three predefined lanes (94, 140, 186 pixels)
- **Pseudo-Random Lane Selection:** Uses BIOS time (INT 1Ah) modulo 3 for lane randomization

7. Scrolling Road Effects

- **Stripe Animation:** Animated road markings using incrementing offset counter
- **Pattern Calculation:** Modulo arithmetic for alternating stripe visibility
- **Frame-Rate Control:** Speed delay counter (`GAME_SPEED_DELAY = 2`) for consistent gameplay

8. Fuel Depletion Mechanic

- **Frame Counting:** 60-frame timer (approximately 1 second) for fuel decrease
- **Visual Feedback:** Color-coded fuel bar (Green > 50, Yellow 25-50, Red < 25)
- **Game Over Condition:** Automatic loss when fuel reaches zero

9. Score System

- **Decimal Rendering:** Custom number-to-string conversion using division by 10
- **Dynamic Display:** Real-time score updates on coin collection (+10 points)
- **UI Positioning:** Fixed-position HUD elements (Score at Y=5, Fuel at Y=5)

10. Persistent High Score System

- **DOS File I/O (INT 21h):**
 - Function 3Ch: Create file with zero attributes
 - Function 3Dh: Open file for reading
 - Function 3Fh: Read data from file
 - Function 40h: Write data to file
 - Function 3Eh: Close file handle
- **Binary Data Storage:** Stores 2-byte score + 20-byte name in `HIGHSCOR.DAT`
- **Automatic Save:** Writes to disk only when current score exceeds high score
- **Error Handling:** Graceful fallback when file doesn't exist

11. Player Identity System

- **Text Mode Input:** Temporary switch to Mode 03h for name entry
- **Keyboard Buffer Management:** Character-by-character input with backspace support
- **Input Validation:** Accepts ASCII 32-126, enforces 19-character limit
- **Default Fallback:** Sets name to "Player" if left blank
- **Welcome Screen Integration:** Displays personalized greeting on title screen

12. Visual Effects

- **Animated Spark System:**
 - 3-frame explosion animation on crash
 - Timer-based frame progression (4 ticks per frame)
 - Synchronized with crash sound effect
- **Enemy Fade-In:** Two-state system prevents premature collision detection
- **Environmental Decoration:** Static grass sprites (trees, flowers, rocks) for visual depth

13. Pause Menu System

- **State Suspension:** Freezes all game logic while preserving state
- **Modal Overlay:** Draws bordered dialog box over frozen game screen
- **Binary Choice Input:** Y/N key detection for "End Game?" prompt
- **ESC Resume:** Returns to gameplay with full state restoration
- **Screen Refresh:** Redraws entire game scene after unpause

14. End Screen & Restart System

- **Results Display:** Shows final score, fuel remaining, and current high score
- **New High Score Announcement:** Special message when record is broken
- **Dual Exit Options:**
 - ESC: Exit to DOS
 - Any key: Full game restart with state reset
- **Complete Reset Function:** Clears all entity arrays, timers, and player data

15. Memory Management

- **Segment Register Usage:**
 - ES = 0xA000 for video memory access
 - CS/DS separation for code and data
- **Stack Preservation:** push/pop discipline in all subroutines
- **Efficient Addressing:** Array indexing with `shl bx, 1` for word-sized data
- **Fixed Memory Layout:** Compile-time ORG directive (`org 0x0100`) for COM format

16. Interrupt Service Routine (ISR) Design

- **Context Preservation:** Saves all registers (AX, BX, CX, DX, SI, DI, DS, ES) on ISR entry
- **Tick Divider:** Executes music logic every 3 timer ticks to reduce CPU load
- **Chained ISR:** Calls original timer handler to maintain system timing
- **Far Jump:** Uses `jmp far` to invoke original interrupt vector

Technical Specifications

- **Language:** x86 Assembly (NASM/MASM syntax)
- **Platform:** DOS (16-bit Real Mode)
- **Video Mode:** VGA Mode 13h (320x240x256)
- **File Format:** COM executable
- **Memory Model:** Tiny (single 64KB segment)
- **Audio:** PC Speaker (programmable timer channel 2)

Assembly Language Concepts Demonstrated

- BIOS/DOS interrupt services (INT 10h, 16h, 1Ah, 21h)
- Direct hardware port I/O (`in`, `out` instructions)
- Segment:offset addressing
- Interrupt vector table manipulation
- Fixed-point arithmetic (no FPU)
- Bitwise operations (`shl`, `and`, `or`, `test`)
- Conditional and unconditional jumps
- Subroutine calling conventions

- String operations (lodsb, stosb, rep movsb)
- Memory-mapped I/O for video
- Timer-based game loops

Game Controls

- **Arrow Keys:** Move player car (Up/Down/Left/Right)
- **ESC:** Pause game or exit from title/end screen
- **Y/N:** Confirm/cancel game end in pause menu
- **Any Key:** Start game from title screen, restart from end screen

Gameplay Mechanics

- Avoid red enemy vehicles (instant game over on collision)
- Collect yellow coins (+10 score)
- Collect orange fuel canisters (+20 fuel, max 100)
- Fuel depletes over time (1 unit per second)
- Game ends when fuel reaches zero or player collides with obstacle
- High score persists across game sessions

Created by Ali Asjad and Chaudhry Ibrahim